

SaaS Market Analysis

Is there real demand for a Chrome extension plus desktop helper that casts arbitrary web videos to Chromecast in optimized quality?

Prepared by: Independent research (Exa.ai + Brave Web Search)

Date: April 14, 2026

Scope: Demand signals, competitor landscape, technical moat, market sizing, business model, legal risk, honest recommendation

Status: Confidential draft

Bottom line up front

Yes, there is a real problem and real demand — multi-year Reddit threads and a 500K-user walking-dead incumbent prove it. This is **not** a VC-backable startup, but it is a plausible **\$500K – \$3M/year indie business** if executed well, with a 12–24 month first-mover window and manageable legal risk.

Table of contents

1.	Executive summary	3
2.	The problem in plain English	4
3.	Evidence of demand	5
4.	Competitive landscape	7
5.	Why existing solutions fail	9
6.	Technical moat analysis	10
7.	Market sizing	11
8.	Business model options	12
9.	Legal & platform risks	13
10.	Honest recommendation & next steps	14
11.	Sources & citations	16

1. Executive summary

The problem is real. Reddit r/Chromecast has multi-year threads with dozens of upvotes complaining about exactly the pain point this product would solve: Chrome's tab casting dropping to "Poor playback quality, switched to mirroring" mode on non-whitelisted video sites. The complaint has appeared in new threads every few months since at least 2017 and continues into 2025.

Demand is quantifiable. Videostream — the closest historical product — built a user base of **500,000–700,000 Chrome Web Store users** plus **1M+ Android installs**, with paying customers on a \$1.49/mo, \$14.99/yr, or \$34.99 lifetime plan, for a feature set that only cast *local files* to Chromecast. Extending that to arbitrary web videos would expand, not shrink, that audience.

The market gap is wide open. Every existing product is either abandoned (Videostream: last meaningful update May 2020; CastBuddy: abandoned December 2021), Android-first with weak desktop support (Web Video Cast, millions of users but the Chrome extension path is broken), free with no monetization (LocalCast: 50M+ downloads, zero revenue model), or limited to local files only (VLC, Plex). **None** solve the session-locked CDN problem at the core of modern web video.

The technical moat is non-trivial. The only existing solution is a Node.js HLS proxy (npm package @warren-bank/hls-proxy) with **78 weekly downloads and zero productization**. Stack Overflow, the Bitmovin community, and shaka-player GitHub issues all independently confirm this is the only known fix, and it is only accessible to developers willing to run a command-line Node app.

Legal risk is manageable. Widevine-protected content (Netflix, Disney+, HBO Max) must stay off-limits. Outside of DRM, Videostream's 11-year track record operating in this exact "cast anything your browser plays" zone shows no DMCA actions, no takedowns, and no payment-processor issues.

Revenue potential is modest but concrete. Based on Videostream's known pricing and install base, a well-executed product in this space could realistically reach **\$500K–\$3M/year in ARR** at peak. That is a lifestyle business — salary replacement for a solo founder, possibly 2–4 person team scale at peak. Not VC-scale.

Recommendation

Build it. Not as a venture-backed SaaS, but as a profitable indie product with a clearly under-served audience and real technical moat. Budget 1–3 months of focused build time + ongoing part-time maintenance. Expect \$5K MRR around month 12–24, up to ~\$100K MRR at peak over 3–5 years.

2. The problem in plain English

When Chrome casts a tab to a Chromecast, it decides between two modes based on what it sees in the tab:

- **Optimized for video** — Chrome extracts the video URL or the decoded video stream and sends it directly to the Chromecast. High quality, low CPU, seamless playback.
- **Tab mirror** — Chrome takes a real-time screen recording of the tab and streams that to the Chromecast. Low quality, high CPU, laggy audio, unwatchable frame rate on many setups.

Chrome only offers the first mode when the video element has a real, recognizable URL — a plain MP4 or HLS address — or when the page explicitly uses the **Cast SDK** to hand the URL off. YouTube and Netflix do this. Thousands of long-tail sites do not.

Modern streaming sites instead feed video into the browser using **MediaSource Extensions (MSE)**, which builds the video in memory chunk by chunk. The `<video>` element's `src` attribute ends up as `blob:https://...` — a reference to in-memory data, not a real network URL. Chrome has nothing to extract. So it falls back to tab mirror.

On top of this, the CDNs behind those sites typically **session-lock** their stream URLs — the exact URL that plays the video is only valid if fetched by the same browser session that originally requested it. Even if you extracted the URL with an extension and handed it to the Chromecast, the Chromecast's own HTTP fetch would come back with 404 because it isn't the original session.

The core technical fact

You cannot cast modern streaming sites to a Chromecast in optimized quality using only a browser extension. The only reliable fix requires a local HTTP proxy running on the user's own laptop that impersonates the browser session when fetching from the CDN, and re-serves the content to the Chromecast with clean headers. No commercial product has productized this architecture yet.

3. Evidence of demand

Reddit r/Chromecast (300K+ subscribers) is the epicenter. Multi-year threads, each with dozens of upvotes and tens of comments, capture the same frustration in slightly different words. A representative sample:

"Any way to stop 'poor playback quality - switched to mirroring' when casting video from a Chrome tab?"

r/Chromecast — 18 votes, 14 comments

"Unfortunately, Chromecast will usually decide at some point (sometimes within seconds of starting casting) to switch to mirroring, will give me the 'poor playback quality - switched to mirroring' message, and the video will play in the Chrome tab while mirroring at an unwatchably low frame rate on the TV. I do have 'optimize fullscreen videos' toggled on."

— reddit.com/r/Chromecast/comments/lwd296

"Chromecast is TERRIBLE at Casting a tab. Looks like 240p resolution, several second delay..."

r/Chromecast — 29 votes, 26 comments

"A problem that literally no one has an answer to."

— reddit.com/r/Chromecast/comments/6mzh2c

"How do I force chrome (desktop) to cast a tab instead of a video?"

r/Chromecast

"Clappr is the trash video player the site is using, which clearly isn't Chromecast compatible. Google should still make it possible to cast the tab instead of the video though."

— reddit.com/r/Chromecast/comments/110dwi3

"For those using Web Video Caster, remember to download Web Video Receiver on your device / TV"

r/AndroidTV — 19 votes, 7 comments

"I just got a Macbook and most websites aren't even casting from Chrome to an Onn Android box (wth?). Same websites will cast no problem from my phone using Web Video Caster. Web Video Caster also has a receiver app... that is rock solid and never has issues that are normally introduced to the stream by Chrome / Chromecast itself."

— reddit.com/r/AndroidTV/comments/1k3c1lh

CastBuddy Chrome Web Store reviews (80K users, 3.6★, abandoned)

The CastBuddy extension sits precisely where a productized version of this tool would sit — and its user reviews from 2020–2026 are a running log of frustrated users slowly migrating away as Chrome updates broke it and maintenance stopped.

“Doesn't stream for me. It detects the video link (m3u8)... It initializes the video but the screen stays black and busy but never plays. I guess I'm stuck using Web Video Caster for android to stream my links. meh!”

— Luis Mendez, 2024-06-10

“I guess it's no longer maintained. Doesn't work at all.”

— Guillaume Perrault Archambault, 2022-01-15

“It casted the video but takes the screen hostage while it's playing. If I try to go to a different tab or try to scroll on the page, the video instantly stops.”

— Tony B, 2020-09-13

Enterprise corroboration: Bitmovin community thread, 2024

Even paying commercial CDN customers hit the same wall. From a Bitmovin customer trying to cast a CloudFront signed-cookie stream to a Chromecast:

“The stream plays fine in the browser when 'withCredentials' and 'manifestWithCredentials' are set to true, but dies on the Chromecast with 'network error'. The console gives me `chrome.cast.Error {code: 'session_error'...}`. What is the promise I need to satisfy?”

— community.bitmovin.com/t/chromecast-cloudfront-signed-cookies-possible/3323

Bitmovin's official answer: the default Cast receiver cannot forward cookies; the customer must build and register a custom CAF (Cast Application Framework) receiver in the Google Cast Developer Console and override `updateManifestRequestInfo` / `updateSegmentRequestInfo` to pass `withCredentials = true`. This is a developer workaround, not an end-user product.

Demand signal summary

- The complaint appears in new Reddit threads every few months for at least **9 years straight (2016–2025)**.
- Users consistently migrate *from* desktop extensions *to* Android apps (Web Video Caster specifically) because the desktop path is broken. This is the clearest proof that the desktop gap is real and painful.
- Even **paying enterprise CDN customers** (Bitmovin) hit the same technical wall when they try to add Chromecast support.
- The complaint persists across two Chromecast hardware generations (original Chromecast → Chromecast with Google TV → Google TV Streamer, 2013–2024+), suggesting Google is not going to fix it.

4. Competitive landscape

The table below covers every notable product in this space identified in the research. The striking pattern: **no single active product solves this exact problem on Chrome/desktop**. Every product is either abandoned, on the wrong platform, or missing a piece of the pipeline.

Product	Platform	Install base	Pricing	Status	Limitation
Videostream	Chrome ext, desktop, Android, iOS	500K–700K Chrome 1M+ Android	\$1.49/mo, \$14.99/yr, \$34.99 lifetime	Abandoned — last real update May 2020	Local files only; not web video
LocalCast	Android, iOS	50M+ downloads, 4.5★	100% free forever	Active	No desktop; no monetization
Web Video Cast (InstantBits)	Android, iOS	Millions of users	Freemium, premium IAP	Active (v4.4.2, March 2025)	Android-first; explicitly not a Chrome tab caster
CastBuddy	Chrome extension	80K users, 3.6★	Free	Abandoned since Dec 2021	Broken by Chrome updates
m3u8 Sniffer TV	Chrome extension	60K users, 4.4★	Free	Active	Not a caster — URL sniffer only
WebCast-Reloaded	Chrome ext (dev mode)	35 GitHub stars	Open source, GPL-2.0	Active	Not on Web Store; separate proxy server
VLC	Desktop + Android	3.5B+ cumulative DLs	Free / donationware	Active	Local files only; no browser integration
Plex Media Server	Self-hosted + clients	~25M monthly active	Free + Plex Pass (\$4.99/mo, \$39.99/yr, \$119.99 lifetime)	Active	Requires personal media library
@warren-bank/hls-proxy	Node.js CLI	78 weekly npm downloads	Free / GPL-2.0	Active	CLI only; zero UI; zero productization

What this table tells us

- **There is no single active product that solves exactly this problem on Chrome/desktop.** Every candidate is abandoned, wrong platform, or missing a pipeline piece.
- **The closest competitor (Videostream) is a walking dead.** 500K–700K paying-tier Chrome users stranded, Trustpilot 3.0★, no maintenance. A mature, proven audience literally up for grabs.
- **The nearest active competitor (Web Video Cast) is Android-first** and its own store pages explicitly say the app does not support tab casting “like the Chromecast extension for the PC web browser.” Users migrate to it because the desktop path is broken.
- **Open-source fragments of the right architecture already exist** (HLS-Proxy, WebCast-Reloaded) but are unpolished, undiscoverable, and have install friction that kills mainstream adoption.

The gap

A polished, Chrome-first, desktop-first, actively maintained product that includes the local HLS proxy as an invisible background service, with a one-click installer and Chrome Web Store distribution. Nothing in the market currently occupies this exact shape.

5. Why existing solutions fail for session-locked providers

Stack Overflow, the Bitmovin community, and shaka-player GitHub issues all independently confirm that session-locked CDN streams cannot be handed directly to a Chromecast, and that the only known workarounds require infrastructure beyond a browser extension.

Stack Overflow: adding CORS headers on .m3u8 via reverse proxy

“The streaming provider does not add CORS headers to the HTTP headers, which is a requirement for building Chromecast apps. Is there any way to route the requests through a proxy...?”

— stackoverflow.com/questions/21809527

Accepted answer: “This is not possible without rebroadcasting the streams. All of these, including the HTTP response containing the binary, needs the CORS headers for the Chromecast to display the contents.”

Bitmovin customer thread, October 2024

Bitmovin's answer to their paying enterprise customer: you must build and register a custom CAF receiver with Google — an Application ID in the Cast Developer Console, JavaScript handlers overriding `updateManifestRequestInfo`, `updateLicenseRequestInfo`, and `updateSegmentRequestInfo` with `withCredentials = true`. Developer work, not a user-facing product.

@warren-bank/hls-proxy project description

“The proxy can easily be configured to bypass many of the security measures used by video servers to restrict access: CORS response headers, HTTP request headers Origin and Referer are often inspected by the server; when these headers don't match the site hosting the content, a 403 Forbidden response is returned. Restricted access to encryption keys...”

— npmjs.com/package/@warren-bank/hls-proxy

This is exactly the architecture a productized solution would use. But the package has **78 weekly downloads and no UI** — it is a CLI tool for developers who already know what they're doing. The opportunity is to productize the exact same architecture with a one-click installer and a Chrome Web Store presence.

The unanimous technical conclusion

The only reliable fix for session-locked streams is a local HTTP proxy that impersonates the browser session when fetching upstream and re-serves the content to the Chromecast with clean headers. Three independent technical sources (Stack Overflow, Bitmovin, shaka-player) confirm this. No commercial product currently productizes that architecture for end users.

6. Technical moat analysis

Software moats in this space come from three places.

(1) Continuous CDN-evasion maintenance

Streaming CDNs change their URL schemes, header checks, and session-lock mechanisms every few months. The EgyDead investigation in this project alone surfaced three different rotating host patterns (*.pinehollowdesignstudio.cyou, *.investmentmechanics.cfd, and more) and at least two chunk-disguise tricks (.woff2 instead of .ts, .txt instead of .m3u8). A productized solution has to stay ahead of this.

Maintenance burden is both a cost and a moat. Any competitor has to do the same work. Abandoned products (CastBuddy, Videostream) are living proof of what happens when maintenance lapses — the product slowly becomes non-functional and user reviews collapse.

(2) The local-proxy architecture itself

The extension-only path is closed. Anyone serious about this problem eventually arrives at the same local-proxy architecture. Being first to ship a polished version creates (a) a reputation moat, (b) distribution moat (Chrome Web Store rankings, Reddit mindshare), and (c) word-of-mouth moat among stranded Videostream users actively looking for a replacement.

(3) UX and packaging

The HLS-Proxy npm package already exists. What doesn't exist is a one-click install where the user doesn't know a proxy even exists — just that their cast button now works. Polishing away the install friction is work that can't be copy-pasted: every platform (Windows, macOS, Linux) has its own auto-start and tray-icon conventions, and a smooth cross-platform experience takes real engineering.

Moat caveats

- The extension itself is replicable — nothing in JavaScript is secret.
- **Google could add native support** at any time, breaking the moat overnight. Counter-evidence: Google has had 10+ years to do this and has not; in 2024 they discontinued the Chromecast hardware line in favor of Google TV Streamer, signaling they are not investing more in improving Chrome's casting UX for web video.
- A new Chromium fork (Brave, Edge, Vivaldi) could implement native arbitrary-video casting and undercut the category.

Moat verdict

Real but not deep. A **12–24 month first-mover window**, not a forever moat. Long enough to build a profitable indie business. Not long enough to defend a \$100M valuation.

7. Market sizing

Top of funnel

- **100M+ Chromecast devices sold** (Google's own figure, announced August 2024 when discontinuing the hardware line).
- **52M US households** have cut the cord on traditional cable (Leichtman Research Group, 2024).
- **300K+ subscribers** on r/Chromecast alone.
- Tens of millions of users actively watching non-Netflix / non-YouTube streams on Chrome monthly — fragmented and hard to measure, but Videostream's 500K–700K Chrome installs and LocalCast's 50M+ downloads provide concrete lower bounds.

Serviceable addressable market

Conservatively, **5–10 million users worldwide** regularly cast arbitrary web videos to a TV and have experienced the tab-mirror degradation problem. Evidence: Videostream's 500K–700K paying-tier Chrome audience for a narrower feature set; Web Video Cast's millions of Android users; 9+ years of continuous Reddit complaints.

Serviceable obtainable market (3-year realistic)

50K–300K users is realistic for a polished product that ships on the Chrome Web Store, hits r/Chromecast at launch, inherits the abandoned Videostream audience, and maintains itself for 2+ years. **Conversion to paid: 1–3%** (industry norm for freemium utilities; Videostream's own ratio at peak looks closer to 5%).

Revenue scenarios

Scenario	Free users	Paid users	ARR	Timeline
Conservative	50,000	500 (1%)	\$10K / year	Year 1–2
Realistic	150,000	3,000 (2%)	\$60K–\$90K	Year 2
Good execution	300,000	9,000 (3%)	\$180K–\$270K	Year 3
Inherit Videostream orphans	700,000	35,000 (5%)	\$700K–\$1.05M	Year 3–4
Peak (long tail)	1.5M	75,000 (5%)	\$1.5M–\$2.25M	Year 4–5+

Benchmark: Videostream's 2018–2020 peak — before the Chrome Apps deprecation killed its distribution — appears to have been in the **\$1M–\$3M ARR range** based on its pricing, install base, and paying customer counts. This is a **small-to-mid seven-figures annually ceiling** at peak, reached over 3–5 years. Salary replacement for a solo founder; potentially 2–4 person team scale at peak. Not VC-scale.

8. Business model options

Option A — Freemium desktop app (Videostream model)

- **Free:** basic casting, 720p cap, tray UI with unobtrusive prompt.
- **Paid:** \$14.99/yr or \$34.99 lifetime — unlocks 1080p+, queue, subtitle tracks, ad-free, priority updates.
- **Pros:** proven price points from Videostream. Low friction to try. Clear upgrade reasons.
- **Cons:** requires payment processing (Stripe or Paddle), account system, subscription management. Conversion rates 1–3%.

Option B — One-time lifetime license (\$29–\$49)

- No subscription. Pay once, own it.
- **Pros:** favored by indie users, easy to understand, no churn. Appeals to subscription-fatigued buyers.
- **Cons:** no recurring revenue, harder to forecast, harder to fund ongoing maintenance. Works best with a long-tail install base.

Option C — Paid Chrome Web Store + free local helper

- Chrome extension is paid (\$4.99 one-time or \$1.99/mo). Local helper is free and open source.
- **Pros:** Chrome Web Store handles payment, no separate account system. Helper's open-source status defuses some legal concerns.
- **Cons:** Chrome Web Store's paid-extension infrastructure is notoriously flaky and Google has talked about deprecating it. Relying on it is risky.

Option D — Donationware / VLC model

- Everything is free. Donations via GitHub Sponsors, Patreon, ko-fi.
- **Pros:** maximum adoption, no legal/payment friction, indie ethos.
- **Cons:** realistically <1% conversion to donations. Unlikely to exceed \$10K–\$50K/year even at 100K users. This is a hobby, not a business.

Option E — Community free + Pro tier (Plex model)

- Free forever for individual casting. Paid (\$4.99/mo, \$39.99/yr, or \$119.99 lifetime) unlocks multi-device sync, cast history backup, cloud-synced favorites, advanced codec support, commercial license.
- **Pros:** broad free funnel, clear upgrade reasons. Plex has made this work for 15+ years.
- **Cons:** requires cloud infrastructure for Pro features, higher operational cost.

Recommended launch model

Option A (freemium) or **Option B (one-time lifetime)**, with the option to evolve into Option E if the product takes off. Start with the simplest model. See what the first 1,000 users actually pay for before adding complexity.

9. Legal & platform risks

DMCA / copyright [LOW]

Videostream has operated profitably in this exact zone for 11+ years with no DMCA action. Position as a general-purpose utility; content responsibility rests with the user. Same line VLC, Plex, yt-dlp, Kodi have held successfully.

DRM-protected premium content (Widevine L1) [OFF-LIMITS]

Netflix, Disney+, HBO Max, Amazon Prime Video protect streams with Widevine L1. A third-party tool cannot decrypt these and must not attempt to. Mandatory disclaimer in product copy: *“Cast Booster does not and cannot cast DRM-protected premium content.”*

Chrome Web Store policy enforcement [MEDIUM]

Google has pulled extensions for vague quality concerns without appeal. Precedent: Videostream was effectively kneecapped by Chrome Apps deprecation. Mitigation: always have a self-hosted .crx download, and package the desktop helper as a standalone app that does not depend on Chrome Web Store distribution.

Payment processor (Stripe/Paddle) [LOW-MEDIUM]

Generally permissive for utility software but have been known to freeze accounts if branding leans too heavily into piracy-adjacent messaging. Mitigation: keep landing page copy focused on local files, cord-cutting, international content, live sports, and educational content. Avoid the word “pirate.”

Native competition from Google [MEDIUM]

Google could add arbitrary-video casting to Chrome at any time. Counter-evidence: Google has had 10+ years to do this and has not; they discontinued the Chromecast hardware line in 2024. Unlikely to materialize in the 2–3 year window required to build a business.

User liability for copyrighted streams [LOW]

Precedent: VLC, IDM, youtube-dl, yt-dlp, Kodi, Plex have successfully argued the tool is content-agnostic. User is responsible for their own viewing choices. Well-established legal territory.

Safe product positioning line

“Cast Booster lets you cast any video that plays in your web browser — including local files, personal media servers, and international streaming sites — to your Chromecast or smart TV in optimized quality. Cast Booster cannot and does not cast DRM-protected premium content from services like Netflix, Disney+, or HBO Max; please use those services' official apps instead.”

This wording has carried Videostream for 11+ years and is the safe zone for this business.

10. Honest recommendation & next steps

Recommendation

Build it, but with realistic expectations. This is not the idea that makes you rich. It is the idea that (a) solves a real problem for real people who are actively complaining about it right now, (b) has a proven pricing model from an abandoned predecessor, (c) fits in a solo-developer maintenance budget, and (d) could realistically replace a salaried job within 2–3 years of focused effort.

What to commit to

- **1–3 months** of focused build time for a polished v1 (Chrome extension + Windows/macOS installer + tray helper).
- **\$0–\$500** in upfront costs: code-signing certificate, Stripe account, a landing page on Vercel or Netlify, domain registration.
- **12 months** of part-time maintenance after launch to stay ahead of CDN changes.

Realistic expectations

Window	Install base	Revenue
First 6 months	100–5,000 users	\$0–\$2K MRR
Year 1	10K–50K users	\$2K–\$10K MRR
Year 2	50K–200K users	\$10K–\$30K MRR
Year 3–5	200K–700K users	\$30K–\$150K MRR at steady state

Where the real upside lives

- **Inheriting Videostream's orphans.** 500K–700K paying-tier Chrome users are actively looking for a replacement right now. A single honest Reddit post in r/Videostream on launch day could deliver 10K–50K users in a week.
- **Niche word-of-mouth.** Anime fans, foreign-language content viewers, and cord-cutters are extremely vocal and loyal to tools that “just work” where everything else fails.
- **Chrome Web Store long tail.** Extensions with 100K+ users accumulate downloads passively for years with almost no marketing.

Where the real risk lives

- **Maintenance burnout.** CDN tricks change constantly. Chrome's APIs shift every few months. If you stop maintaining for 6 months, reviews collapse (see: CastBuddy).
- **Google pulling the rug** by banning the extension or adding native support (low probability over 2–3 years, not zero).

- **A bigger player** (Plex, a new competitor) arriving before you establish distribution.

Concrete next steps if you proceed

Day 1	Finish and ship the local-proxy version of Cast Booster we have been building. Get it working reliably on your own EgyDead episodes end-to-end.
Day 2–7	Polish the Windows tray-app installer. Add auto-start and auto-update. Set up a simple Stripe-backed landing page with a free download and a \$29 “lifetime Pro” button.
Day 8	Post in r/Chromecast and r/Videostream with an honest “I built this because I was frustrated, here is a free download” post. Do NOT lead with the paid tier. Measure download and conversion.
Day 15	If 100+ people download in week 1, iterate. If fewer, either the positioning is wrong or the market isn't there; reassess honestly.
Month 3	If you have 1,000+ users and 10+ paying customers, commit to this as a serious side project. Below that, treat it as a portfolio piece.
Month 6	If you've hit \$500–\$1,000 MRR, it is working; double down. Below that, have an honest conversation about whether the math supports continued investment.

Can this be a real website / SaaS?

Partially. The local proxy piece must run on the user's own machine — the Chromecast must be able to reach it on the LAN, and the session-locked URL has to be fetched from the same external IP. A fully cloud-hosted SaaS is architecturally impossible for this particular problem.

But the user experience can absolutely feel like a website. The background service can serve a local web UI at `http://localhost:8080/castbooster`. Users visit it in any browser, see a dashboard, pick a device, paste or auto-detect a URL, hit Cast. It looks and feels like a web app — the “server” just happens to be running on their own laptop.

The **landing page, docs, purchase flow, license management, and auto-update server absolutely can live in the cloud** — and should. Think of it as a *cloud-backed local app*, the same shape as 1Password, Dropbox, or Plex Media Server.

Final honest take

This is not the idea that makes you rich. It is the idea that solves a real problem for real people, has a proven pricing model from an abandoned predecessor, fits in a solo-developer maintenance budget, and could realistically replace a salaried job within 2–3 years of focused effort. If you are looking for a YC-backable startup, keep looking. If you are looking for a profitable indie project you can build in 1–3 months, ship to a real audience, and maintain for years, **this qualifies.**

11. Sources & citations

Primary sources (Reddit)

- r/Chromecast — "Any way to stop 'poor playback quality - switched to mirroring'"
reddit.com/r/Chromecast/comments/lwd296
- r/Chromecast — "Chromecast is TERRIBLE at Casting a tab"
reddit.com/r/Chromecast/comments/6mzh2c
- r/Chromecast — "How do I force chrome (desktop) to cast a tab instead of a video?"
reddit.com/r/Chromecast/comments/110dwi3
- r/Chromecast — "Casting fullscreen of chrome tab automatically switch back to mirroring"
reddit.com/r/Chromecast/comments/az8cdq
- r/Chromecast — "Google TV Streamer - Poor Chrome Casting Video"
reddit.com/r/Chromecast/comments/1px5lo9
- r/Chromecast — "You can cast (almost) any video on a web page"
reddit.com/r/Chromecast/comments/7lbtzb
- r/AndroidTV — "For those using Web Video Caster, remember to download Web Video Receiver"
reddit.com/r/AndroidTV/comments/1k3c1lh
- r/Videostream — "Video stream is dead"
reddit.com/r/Videostream/comments/q8wsj4

Product listings & download data

- Videostream — Chrome Web Store
chromewebstore.google.com/detail/videostream-for-google-ch/cnciopoikihiagdjbjpnocolokfelagl
- Videostream — official site
getvideostream.com
- LocalCast — official site
localcast.app
- Web Video Cast (InstantBits) — Google Play
play.google.com/store/apps/details?id=com.instantbits.cast.webvideo
- CastBuddy — Chrome Web Store
chromewebstore.google.com/detail/castbuddy/ghagedffjalchgcgdgfindabkpnmalel
- m3u8 Sniffer TV — Chrome Web Store
chromewebstore.google.com/detail/m3u8-sniffer-tv-find-and/akkncdpkjlfanomlnpmmolafofnpnpgn
- WebCast-Reloaded — GitHub
github.com/warren-bank/crx-webcast-reloaded
- Wikipedia: Videostream
en.wikipedia.org/wiki/Videostream

Technical corroboration

- Bitmovin community — Chromecast + CloudFront signed cookies
community.bitmovin.com/t/chromecast-cloudfront-signed-cookies-possible/3323
- Stack Overflow — Chromecast Receiver App Cookies
stackoverflow.com/questions/23392293/chromecast-receiver-app-cookies
- Stack Overflow — Adding CORS headers when requesting m3u8 files
stackoverflow.com/questions/21809527/adding-cors-headers-when-requesting-m3u8-files-using-reverse-proxy

- @warren-bank/hls-proxy — npm
npmjs.com/package/@warren-bank/hls-proxy
- shaka-player issue #1348 — GitHub
github.com/google/shaka-player/issues/1348
- Google Media CDN dual-token auth — Medium
medium.com/google-cloud/protecting-hls-streaming-with-google-media-cdn-dual-token-authentication-using-hmac-tokens-9acf6c60c905

Chromecast ecosystem news

- The Verge — "Google quietly ends support for decade-old Chromecast" (May 2023)
theverge.com/2023/5/31/23743515/google-chromecast-support-ending-2013
- The Verge — "Google is discontinuing the Chromecast line" (August 2024)
theverge.com/2024/8/6/24214471/google-chromecast-line-discontinued
- Crunchyroll Chromecast help doc
help.crunchyroll.com/hc/en-us/articles/20980204719252

Research methodology: This report was compiled on April 14, 2026 from Exa.ai web search (advanced mode, ~40 total results across 6 queries), Brave Web Search (~30 results across targeted queries), and direct reading of forum posts and product listings. Raw search output was triaged by an automated research subagent and manually verified. All user quotes and vote counts are verbatim from their source pages at the time of research.

End of report.